

- ① pom.xml
- ② logging level Error/Info/Trace/Debug
- ③ How to create a springboot project
- ④ <dependency> & <exclusion> {embedded, tomcat}
- ⑤ application.properties → change port/logging level
- ⑥ Tomcat, Jetty, undertow
- ⑦ Server v/s Framework
- ⑧ Profiles
- ⑨ AutoConfigure
- ⑩ Spring boot follow MVC Arch — Model
— View
— Controller
- ⑪ Rest SOAP
- ⑫ Post, Get, Put, Patch, Delete unsafe
safe

⇒ Profiles ⇒ Ex: ⇒ Swissy/Zomato. → (Intro Env)

↳ Staging/Development, Pre-Prod, Prod

DB & resource will be diff for diff env. & it's not good to change app. properties file for diff. env. So we can create diff profiles.

→ Create Profiles → {resources folder}

Syntax → {appⁿ-(env).properties} Ex
appⁿ-dev.properties

- Before Profile, how server.port gets automatically mapped. (auto config.)

- We can see server.port in auto config file & default port no.

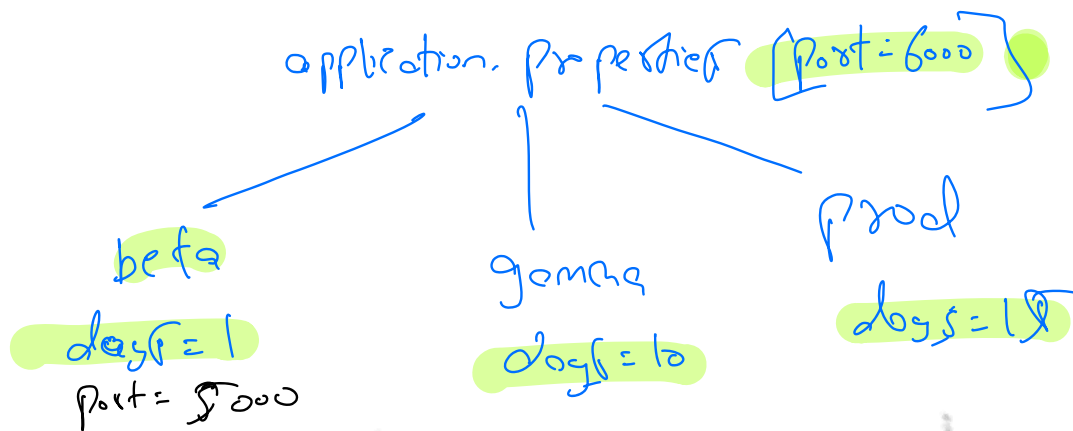
- We can see datasource.url which is used for db connection in app.properties. :->

spring.datasource.url=jdbc:mysql://localhost:3306/test

Property < Spring related {can directly give name & value}
Custom {@value("\${db-url}")}

→ To run profiles we can edit config. :->

spring.profiles.active=game.



Case 1

profile = dev

port = ? (6000)

debug = ? (1)

Case 2 { beta have }
port = 5000

Case 3

if we don't
give port

5000

1

8080

1

⇒ We can even use properties for security like

server.port =

empty

& formatted port but

it will automatically fill

→ We can even put

spring.profiles.active = beta in prop

file

→ Can I put multiple profiles simultaneously,
if yes then who is given the priority?

spring.profiles.active = beta, gamma, prod.

→ Yes & rightmost(profile)

MVC

M → Model
V → View
C → Controller

Ex: → Amazon Prime Video

Model → Data stored in DB
Video {id, name, genre}
User {id, name, email}

Flow →

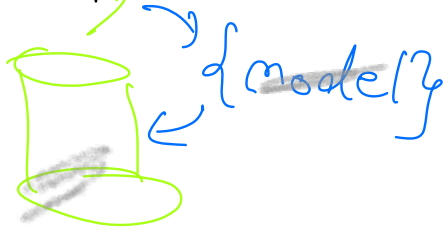
Controller [API endpoint, Validation]

View → (UI wants to show)
{name, genre}

↓
Service [Bus. logic]

Controller → Point where
client's req. lands.
(API)

↓
Repo [Data handling]



Business
logic

↓
Data handling

↓
Calculation

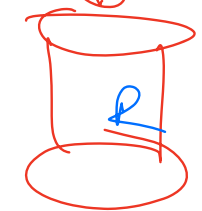
Client

1.) Creating User Request
name:
email:
location:



Server

Model
id
name
email
location
createAt
updateAt



2.) Get
id

{name
email
location}

View / Response DTO

Request & Response are DTOs

Request DTO

Response DTO

not model

→ data transfer objects.
used for interaction b/w Client & Server

Now we start making a real appⁿ.

✗ We shld not relay the info that how model looks like
(Request DTO & Model shld be diff)

✗ For Response DTO

→ Understand who are clients of your API.

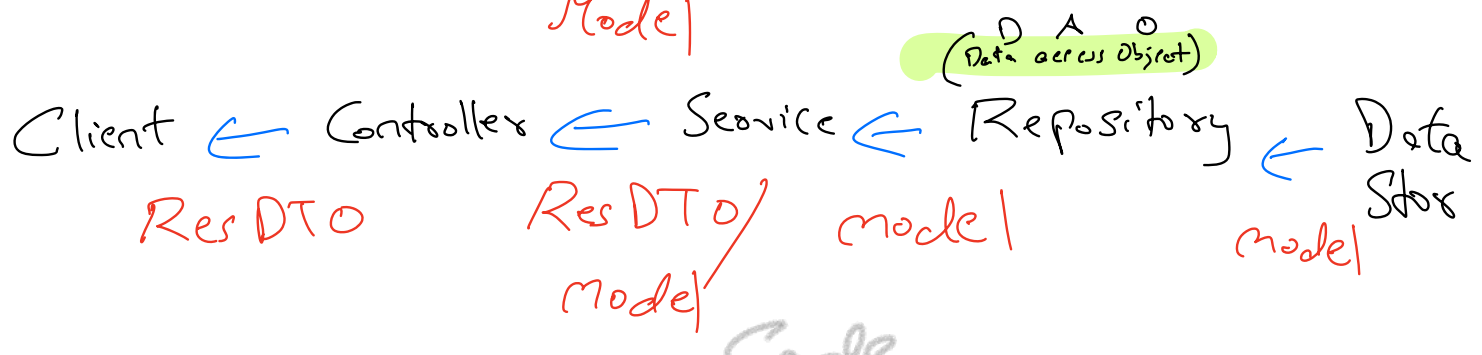
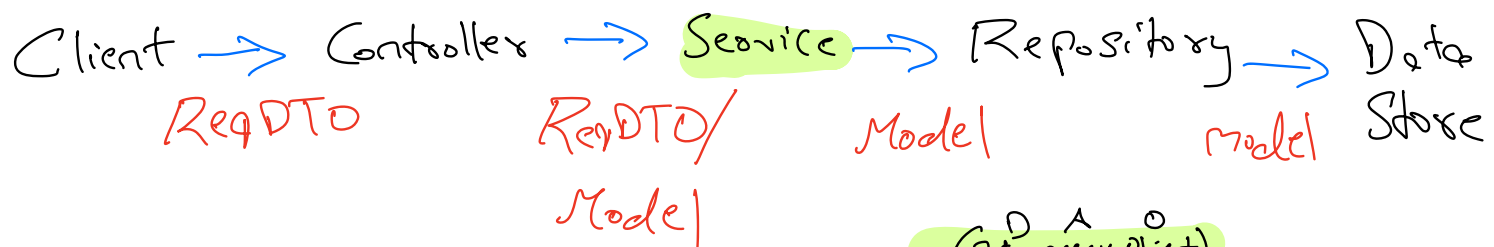
→ Their reqⁿ.

→ Who can be future clients. (hard to speculate)

Don't give all attributes in response thinking of future
as that lead to → ① extra data = extra latency
② query multiple tables unnecessarily

Code

✗ DTO shld not flow to repository layer.



→ When we are creating user, we can return integer which can be used by get API to fetch
So shld we return Integer or a response?

→ We shld return response or :-

① Integer number } Compatibility issue?
(Java) (JS)

→ If we return Java obj, it can be converted to JS

② Scoping as if I want to return more values...

③ Visibility { Manually you have to check response }

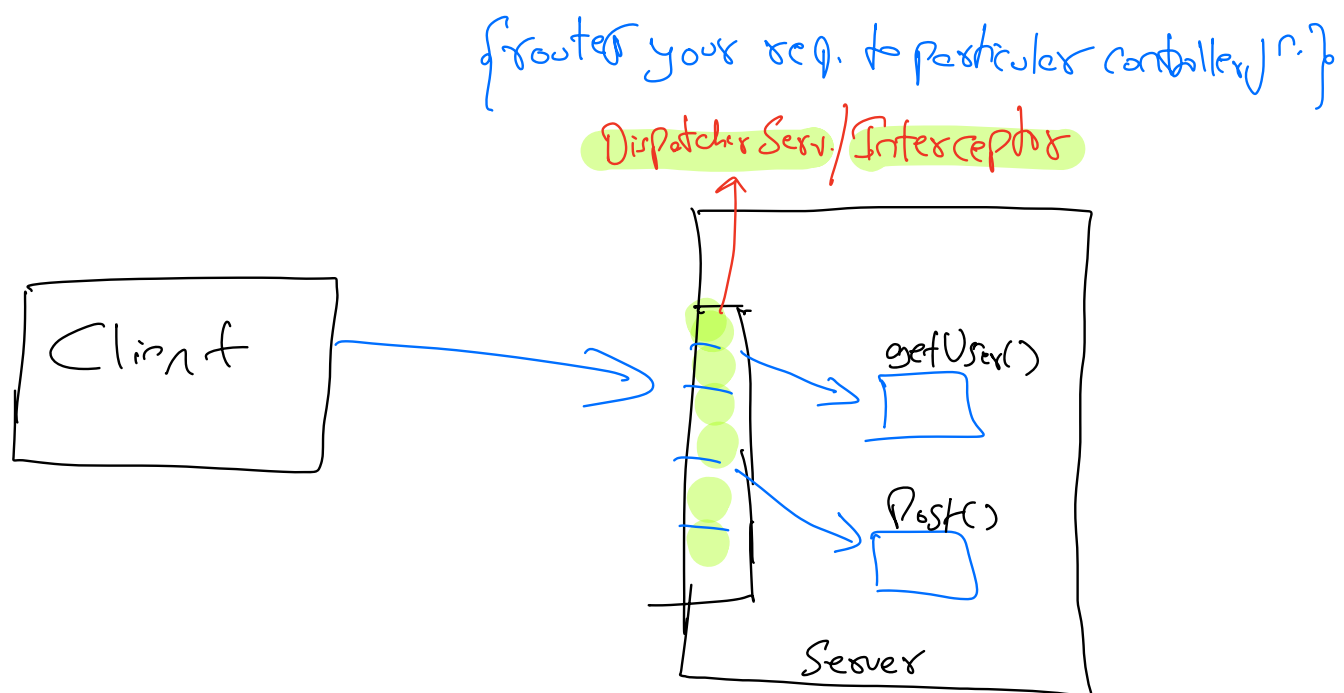
→ Integration → Scoping & extensibility → Visibility

→ Now there are not APIs, there are just normal Java classes.

→ If we don't create API, our code will not get visibility of dispatcher servlet side

→ We need @RestController on Controller.

↓
This makes classes visible to Controller to dispatcher serv.



↳ HTTP Req. Methods → Get, Post, Put, Patch, Delete

2 Comp. Arch → Rest & SOAP

- {stateless}
- {return JSON, XML}

stateful or Stateless

{only XML}

Ex: ① Banking - first

② App. Tracking detection

↳ Basically in SOAP, first it creates a state & then performs that state action.

→ We can hit get API from browser

→ But not others